

Rapport de stage — M1 AM

---

# Exploration du paysage des Réseaux Booléens à Signe Pluripotents

---

Équipe BCM — Laboratoire TIMC, Grenoble

**Quentin Boyer**

Encadrants : Nicolas Glade, Laurent Trilling

Université Grenoble Alpes, Grenoble INP ENSIMAG

Mai-Juin 2026

## Résumé

**Résumé.** [À rédiger en dernier.]

**Abstract.** [English version of the abstract.]

**Mots-clés :** Systèmes biologiques, réseaux booléens à signe (SBN), pluripotence, robustesse, évolutivité, Answer Set Programming, calcul parallèle.

# Table des matières

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>4</b>  |
| 1.1      | Motivations . . . . .                                      | 4         |
| 1.2      | Positionnement dans les travaux de l'équipe . . . . .      | 5         |
| 1.3      | Contributions de ce stage . . . . .                        | 5         |
| <b>2</b> | <b>Formalisme des Réseaux Booléens à Signe</b>             | <b>6</b>  |
| 2.1      | Structure et dynamique . . . . .                           | 6         |
| 2.2      | Fonctions booléennes de seuil (SBF) . . . . .              | 6         |
| 2.3      | Distances, règles de mutation . . . . .                    | 6         |
| 2.4      | Quel paysage des réseaux explore-t-on ? . . . . .          | 7         |
| <b>3</b> | <b>Développement : Méthodes et outils</b>                  | <b>7</b>  |
| 3.1      | Rectitude . . . . .                                        | 7         |
| 3.2      | Efficacité . . . . .                                       | 8         |
| 3.3      | Observation . . . . .                                      | 9         |
| 3.4      | Usage des grands modèles de langage en recherche . . . . . | 9         |
| 3.5      | Exploration Monte-Carlo : MCSBN . . . . .                  | 9         |
| <b>4</b> | <b>Postérité et résultats</b>                              | <b>9</b>  |
| 4.1      | Guide de génération des CSV . . . . .                      | 10        |
| 4.2      | Guide d'utilisation des HTML . . . . .                     | 12        |
| 4.3      | Observations et limites . . . . .                          | 12        |
| <b>5</b> | <b>Conclusion, travaux inachevés et ouverture</b>          | <b>12</b> |

# 1 Introduction

## 1.1 Motivations

Un des intérêts de la modélisation des systèmes biologiques (au sens large, à toute échelle : écosystèmes, cellules, expression génétique, relations symbiotiques, neurones...) est qu'elle permet suffisamment de simplifications et d'abstractions pour pouvoir isoler (clarté) et forger des concepts sur lesquels on peut facilement travailler (au sens mathématique) et dont on peut évaluer la pertinence (ce qui est un objectif en fin de compte, forger des concepts pertinents pour que ceux qui viennent après s'en emparent). L'étude *in vivo* (ou juste du vivant ?) ne peut généralement qu'entrapercevoir ces mêmes concepts qu'au milieu du fouillis des particularités biologiques, et encore, souvent au travers du prisme de la vie telle qu'on la connaît (prisme duquel on voudrait s'affranchir s'il est question d'origine du vivant, voire d'exobiologie), là où le modèle ouvre la voie à l'exhaustivité et à la généralisation (on a bien dit ouvrir la voie, pas résoudre le problème). C'est donc dans ce cadre que nous nous plaçons : l'exploration des horizons d'un modèle (SBN et variations) pour observer les contraintes qu'il impose et les lois que celles-ci font émerger (développer l'idée que contraindre permet ?). Il n'est donc pas question ici d'évaluer la validité biologique du modèle (même si nos réflexions à ce sujet guident nos choix) mais de bien modestement (j'ai deux mois, ce n'est pas énorme) en exploiter le potentiel conceptuel (et l'intérêt porté à cette exploitation est en quelque sorte une mesure de sa validité (i.e. utilité) conceptuelle).

La caractéristique du vivant qui sert de point de départ à ce travail est la pluripotence, c'est-à-dire la capacité d'un système unique à présenter différents comportements qui eux-mêmes correspondent potentiellement à d'autres systèmes (plus petits), on dit que le système principal les "simule". Typiquement l'expression du code génétique (d'où proviendra souvent par analogie le vocabulaire que nous emploierons) dont la spécification des cellules est un bon exemple, des poils de chat court ou long selon la saison, la génération des pensées par nos neurones... [À fournir et préciser].

Le modèle exploré ici est donc celui des Réseaux Booléens à Signe (SBN), formalisé en section [X.] et ce qui nous y intéresse particulièrement, c'est l'interprétation des nœuds du réseau comme une STRUCTURE possédant différents états, et dont l'évolution déterministe génère une DYNAMIQUE dans l'espace du graphe de transition de ces états (hypercube à  $n$  dimensions). [Insérer illustration]. La pluripotence [concept volontairement flou/général pour pouvoir capter différentes interprétations] peut donc prendre sens dans le formalisme des SBN comme la présence de sous-dynamiques stables sur des supports de dimension inférieure reste [Cf. Quel paysage explore-t-on ?].

## 1.2 Positionnement dans les travaux de l'équipe

Le présent travail n'est pas isolé, il s'intègre dans une réflexion plus large menée de longue date par Nicolas Glade sur le vivant, et à laquelle ont participé (et participe) de nombreux chercheurs et étudiants (souvent dans l'équipe BCM du TIMC). Consulter leurs écrits s'est avéré très instructif (le lecteur y est largement invité) et aura permis de "Se tenir sur les épaules des géants et voir plus loin, voir dans l'invisible, à travers l'espace et à travers le temps." - Jean-Claude Ameisen. En voici une liste (non exhaustive, mais rassemblant les principaux).

- HDR de Nicolas Glade : la bible donnant le cadre général [Cf. tout].
- Thèse de Rémi Segretain : réflexions générales sur les SBN et développement de la Java Parallel Pipeline (JPP) [Cf. partie technique]
- Rapports d'Ailing Bonnet et de Rémy Defossez : approfondissement des travaux de Rémi [Cf. partie calcul de propriétés].
- Rapport de Danwen Wu : Démonstration mathématique de l'équivalence entre l'existence de dynamiques séparées sur deux hyper-faces disjointes et de contraintes sur les poids du réseau [Cf. partie Méthodologie d'exploration].
- Programmes de Laurent Trilling : le point de départ de ce travail, l'énumération en Answer Set Programming (ASP) des réseaux pluripotents sous nœud de contrôle via la dynamique [Cf. partie Méthodologie d'exploration].

Pour ce qui est des travaux encore en cours au moment de la rédaction, Corentin Lacaze s'occupe de la simulation d'agents dont les comportements sont intégralement générés par des SBN. L'objectif étant de les mettre en compétition pour observer des phénomènes évolutifs et comprendre comment cela se traduit en termes de SBN. La complémentarité entre l'aspect théorique du présent travail d'un côté et l'aspect expérimental des simulations de l'autre aura alimenté nos échanges qui nous auront permis l'un l'autre de nous guider mutuellement.

## 1.3 Contributions de ce stage

L'apport principal de ce travail est la recherche de la pluripotence dans les SBN. L'enjeu principal a donc été d'abord d'énumérer efficacement les SBN qui nous intéressent, la définition de ce qui nous intéresse étant souvent guidé par l'exigence d'efficacité (recherche de symétries). L'efficacité mémoire et temporelle est cruciale, les espaces de recherche explosent combinatoirement avec le nombre de nœuds du réseau et il n'est en pratique pas raisonnable d'espérer l'exhaustivité à partir de 4 nœuds, où même d'avoir des données représentatives à partir de 5 nœuds [Cf table des échelles]. Une bonne partie du temps de travail a donc été dédié à l'optimisation du code pour essayer de récolter le maximum d'information avec un minimum de calculs.

Dans un second temps l'objectif a été de créer des visualisations utiles et pratiques,

à la fois pour apprécier intuitivement l'espace des réseaux à travers différents angles et différentes métriques, mais aussi pour vérifier la justesse de nos programmes. Cela se matérialise par les trois scripts hmtl/javascript présents à la racine du git, permettant tous les trois de charger des fichiers CSV générés indépendamment et d'en afficher de manière concise le contenu.

- `sbn_viz(7).html` : réseaux et graphes de transitions détaillés.
- `sbn_scatter(5).html` : nuages des réseaux placés selon différentes métriques.
- `metagraphe.html` : nuages des réseaux placés les uns par rapport aux autres selon des règles de mutations.

[Cf. partie technique pour un usage détaillé]

Ce stage aura aussi donné l'occasion de rédiger un article synthétisant une longue réflexion sur les SBN ???

## 2 Formalisme des Réseaux Booléens à Signe

### 2.1 Structure et dynamique

[Cf. rapport Ailing]

### 2.2 Fonctions booléennes de seuil (SBF)

[Cf. rapport Ailing]

+ Table des échelles (Dimension, SBF, produit de SBF, SBN, PSBN, Poids, vecteurs, arbres)

### 2.3 Distances, règles de mutation

Une approche proposée par Rémi (et poursuivie par Rémy et Ailing) a été de construire des méta-graphes dans lesquels deux dynamiques sont reliées si elles sont voisines en termes de distances (pouvant répondre à différentes définitions) et dont la position est calculée par un algorithme de type Force-directed.

Il a été essayé de reproduire les mêmes méta-graphes en réutilisant les définitions de distances, en particulier la distance DA agrégeant les changements des cycles limites. Mais comme le calcul exhaustif de ces distances tel quels ne pouvait être mis à l'échelle pour la dimension 3 qu'aux prix d'efforts conséquents (de l'ordre de  $1e10$  paires), et est difficilement concevable en dimension 4 (de l'ordre de  $4e20$  paires). Il est sans doute possible de trouver des manières de contourner cette difficulté qui n'est que technique, et de construire des méta-graphes similaires sémantiquement. En tout cas cette piste (qui existe) n'a pas été exploré plus avant ici.

Cependant l'idée de construire des méta-graphes Force-directed n'a pas été totalement mise de côté, nous nous sommes juste plus modestement limités à des méta-graphes structurels (ou les dynamiques sont liées si elles possèdent des structures voisines). Cela nous permet notamment de faire un lien direct avec les simulations de Corentin Lacaze, car les règles de mutations qui dirige l'évolution des agents dans ce contexte sont des relations de voisinages entre structures. L'exploration contrainte par l'évolution des agents se traduit alors par un chemin (avec de potentiels embranchements, donc un arbre) dans le méta-graphe, et cela permet de poser des questions du type "Est-ce que certaines propriétés des dynamiques favorise le passage des chemins", "Certaines zones sont-elles plus viables que d'autres? C'est-à-dire est-ce que les chemins préfèrent certaines zones?" "Pour arriver à un phénotype cible, est-ce que plusieurs chemins sont possibles, si oui en quoi sont-ils différents?" .

La règle de mutation considérée en premier a donc été simplement celle d'un changement de poids quelconque (dans les limites de la borne inf des poids permettant d'exprimer toute les dynamiques) sur un seul arc du réseau. [Insérer équation "Mathématiques"] Pour faciliter la réutilisation du code, j'aimerais faire en sorte qu'un utilisateur extérieur puisse facilement modifier ces règles de mutations à l'envie, et régénérer les csv.

## 2.4 Quel paysage des réseaux explore-t-on ?

- Les différents aspects de la pluripotence : [À illustrer et formaliser à chaque fois]
- Noeuds de contrôle -> vecteurs de décomposition complet -> fichiers undercontrol.
- Formalisme des sous réseaux -> fichiers PSBN/PTBN.
- Contrainte supplémentaire -> fichiers PSBNI (pour inhibiteurs).
- La partialité -> vecteurs de décomposition partiel -> non explorée.
- Noeuds contrôlés -> offuscation -> non explorée.

## 3 Développement : Méthodes et outils

Comme souvent dans l'écriture de code, les enjeux des phases de développement ont toujours été dans un premier temps la correction (l'exactitude) puis la terminaison (idéalement rapide). Mais comme la rectitude nécessite surtout de bien savoir ce que l'on cherche, et que pour cela, il faut avoir des visualisations efficaces, les étapes de correction, d'optimisation et d'observation se sont succédé en cycles. Les sections suivantes ne reflètent donc pas fidèlement la chronologie du développement.

### 3.1 Rectitude

L'objectif originel du stage était de prendre en main le langage Answer Set Programming (ASP) pour parvenir à comprendre et améliorer les programmes existant [Cf.

Programmes Laurent dans le git].

Explication/Promo ASP - Clingo - Clingcon - ApproxAsp

Explication du programme de Laurent [Cf. un papier pas vraiment fini]

- travail sur la dynamique
- Visualisation `sbn_viz` pour apprécier. [Cf. fichier du git]
- Génération des réseaux pluripotents à 1 noeud de contrôle, de sous-réseau SBN, d'arbre de décomposition complet fixé, avec hyper-faces distinctes. Calcul lourd et peu scalable (utilisable en temps et ressources raisonnables que jusqu'en dimension 3).

Le rapport de Danwen prouvant qu'il est possible de caractériser cette famille de réseaux en ne travaillant que sur les poids, c'est dans cette direction que nous sommes partis, aboutissant au fichiers :

- `PTBN_undercontrol.lp` permettant de générer des réseaux pluripotents à 1 nœud de contrôle, et d'arbre de décomposition fixé. (Et de sous-réseaux quelconques, donc TBN à cause du contrôle). [explication du fonctionnement précis du code ?]

- `PSBN_undercontrol.lp` extension du fichier précédent permettant de préciser que les sous-réseaux sont bien des SBN.

La propriété d'avoir différentes hyper-faces nous a semblé peu essentielle [préciser] au regard des rendus que l'on voulait afficher par la suite et a donc été laissée de côté. (`PSBN_distinct_faces_HS.lp` a quand même été développé dans l'idée de confirmer que nos résultats correspondaient à ceux des programmes initiaux, ce qui a été le cas pour certaines configurations : les 189  $\langle 0, 2, 0, 0 \rangle$  en  $d = 3$  notamment).

- `PSBN_undercontrol_undecomposable_beyond.lp` et `PSBN_undercontrol_undecomposablePBN_b` [Question de décomposition fines] [Cf. la table des échelles pour pourquoi ce n'est pas si important]

- `PTBN_undercontrol_treeinsolver.lp` prenant en compte n'importe quel arbre de décomposition à vecteur de décomposition fixé. Prise en compte des permutations sur les noeuds.

- `PSBN_undercontrol_treeinsolver.lp` de la même manière qu'avec les fichiers `undercontrol` simple, extension du fichier précédent permettant de préciser que les sous-réseaux sont bien des SBN.

## 3.2 Efficacité

- Base ASP : contraintes sur les poids plus efficaces que les contraintes sur la structure. Parallélisation possible directement en ASP, mais généralement peu efficace [donner des chiffres].

- > Répertoire JPPipeline du git. (Java)

- JPPipeline : parallélisation à plus au niveau permettant de mieux tirer partie des différent coeurs de la machine, et de mieux gérer l'utilisation mémoire.



- Le fonctionnement général.
- Ma pipeline. [Donner une image globale avec des instances à chaque fois].
- L'énumération avec garde-fou.
- utilisation
- post processing python relativement abandonné ici.
- Utilisation machine HPC [Donner les spéc de la machine et des exemples de gains]

### 3.3 Observation

- > fichiers html à la racine du git.
- Beaucoup de questions de Front : lisibilité (palette de couleur, styles des points, fonctions interactives, ordre d'affichage), fluidité (webGL, parsing en streaming, rafraîchissement partiel), ergonomie (parsing des différentes conventions de CSV mises en place pendant le développement)
- Mais aussi beaucoup de questions de fonds : Comment organiser la légende ? Quelles statistiques afficher ? Comment donner le plus d'information possible ? Barres d'erreur ? Quelle décomposition mettre en valeur ?

### 3.4 Usage des grands modèles de langage en recherche

- Possibilités permises
- Usage différencié selon les parties du projet
- Claude code en version pro. Utilisation naïve.

### 3.5 Exploration Monte-Carlo : MCSBN

- > répertoire MCSBN du git. (python et C)
- Tentative en véritable vibe code (autorisation d'écriture sans demande de permission) d'énumération sans passer par l'asp pour voir 1. Si c'est plus efficace 2. Quelles en sont les limites.
- énumération brute puis post processing -> on ne récupère quasiment que des non décomposés.
- énumération d'un coeur pluripotent puis de sa frontière en marche aléatoire (`core_then_walk.py`)
- > illustre bien que la manière de générer est déterminante.
- utilisation de DRGraph pour précalculer le layout Force-directed rapidement.

## 4 Postérité et résultats

Un des objectifs du projet étant son utilisation par autrui, j'essaie de présenter ici les moyens de (~~s'amuser avec~~) s'en servir sans avoir à s'y plonger trop avant.

## 4.1 Guide de génération des CSV

Utilisation de la pipeline :

- Avec VScode utiliser le launch.json ou alors par le terminal [Insérer instructions] - changement d'énumérateur asp : implémenter dans JPPipeline/src/asp, changer l'argument dans le json, et garder les mêmes noms de prédicat (ou inspecter les regexp mais ce n'est pas recommandé) - changement de statistiques

Utilisation de MCSBN :

La génération d'un *paysage hybride* (cœur des dynamiques les plus décomposées, puis voisinage connexe) se fait en deux temps : génération du CSV, puis calcul d'un *layout* pour la visualisation. Tous les chemins sont relatifs au répertoire MCSBN/.

# 1. générer le paysage (cœur exhaustif + frontière par marche)

```
python src/python/core_then_walk.py -d 4 -M 20000 -n 80000 \
    --min-leaves 8 --fanout 8 -o paysage.csv
```

# 2. ajouter le layout DRGraph (colonnes metagraph\_x, metagraph\_y)

```
bin/metagraph_layout paysage.csv -o paysage_meta.csv
```

# 3. charger paysage\_meta.csv dans metagraphe.html

**Génération du paysage (core\_then\_walk.py).**

-d dimension du réseau (nombre de nœuds). Détermine la taille de l'espace (370 SBF distinctes en  $d=4$ , 11 292 en  $d=5$ ...).

-M taille du *cœur* (phase 1) : on conserve les  $M$  dynamiques les plus décomposées (plus grand *decompSum*), couvertes exhaustivement.

-n nombre total de dynamiques du paysage final; la phase 2 explore le voisinage connexe du cœur jusqu'à atteindre  $n$  ( $n - M$  nœuds de voisinage).

-o fichier de sortie (- pour la sortie standard; suffixe .gz pour une compression à la volée).

--min-leaves profondeur minimale des arbres de contrôle énumérés en phase 1 (un arbre à  $L$  feuilles = décomposition en  $L$  morceaux). Plus élevé  $\Rightarrow$  cœur plus petit et plus profond; plus bas  $\Rightarrow$  dynamiques moins décomposées, plus nombreuses (plus lent). Défaut : 2 si  $d \leq 3$ , sinon 8.

--max-free nombre maximal de nœuds *libres* par arbre (borne le coût). 0 (défaut en  $d \geq 4$ ) = arbres entièrement contrôlés, ce qui est *complet pour le cœur profond* et rapide; l'augmenter inclut des classes moins décomposées. Illimité en  $d \leq 3$ .

- `--fanout` nombre de voisins nouveaux pris par nœud à chaque couche de l'expansion de frontière (phase 2). Grand ( $\geq 8$ ) = pourtour épais collé au cœur ; petit (1–2) = couches fines, exploration plus lointaine. Défaut 4.
- `--core-bin` backend de la phase 1 : `auto` (défaut) utilise le binaire C `bin/mcsbn` s'il existe ( $\sim 45\times$  plus rapide), sinon l'énumérateur Python ; `none` force Python.
- `--seed` graine du générateur aléatoire (reproductibilité de la phase 2).
- `--measure` loi d'échantillonnage : `variety` (uniforme sur les dynamiques distinctes) ou `genotype` (pondérée par le nombre de matrices de poids réalisant chaque dynamique).
- `--no-weights` omet les colonnes `f_*` et `w_*` (CSV plus compact, mais incompatible avec le métagraphe qui a besoin des `f_*`).

La phase 1 peut aussi être lancée seule par le binaire C,  $\sim 45\times$  plus rapide (réutilisé automatiquement par `core_then_walk.py`) :

```
bin/mcsbn -d 4 --core-trees --min-leaves 8 -M 20000 -o coeur.csv
```

**Layout DRGraph** (`metagraph_layout`). Construit le métagraphe (nœuds = dynamiques, arêtes = mutations voisines), appelle DRGraph et ajoute les colonnes `metagraph_x`, `metagraph_y` (positions 2D) afin que `metagraphe.html` affiche le paysage sans recalculer de layout. Le binaire se construit une fois par `make drgraph` (qui détecte GSL/BOOST dans l'environnement conda).

- (*positionnel*) le CSV d'entrée (ou `.csv.gz`) ; doit contenir les colonnes `f_*`.
- `-o` sortie annotée (par défaut un nom dérivé de l'entrée).
- `--stats-only` n'écrit rien : rapporte seulement la structure en composantes connexes (utile pour vérifier que le métagraphe percole avant de lancer le layout).
- `--vis` chemin du binaire DRGraph Vis (défaut : `tools/DRGraph/Vis` ou la variable `$MCSBN_VIS`).
- `--samples` nombre d'itérations d'optimisation DRGraph (le temps y est proportionnel ; défaut 400, 150–200 suffisent souvent).
- `--neg` nombre de *negative samples* par pas de gradient (force de répulsion approximée ; défaut 5).
- `--gamma` poids de la répulsion : grand  $\Rightarrow$  nœuds plus écartés, petit  $\Rightarrow$  amas plus serrés (défaut 0,1).
- `--A`, `--B` paramètres  $a$  et  $b$  de la fonction de proximité ;  $b$  contrôle la forme ( $b=1$  amas locaux,  $b=2$  équilibré (défaut),  $b=3$  longueurs d'arêtes uniformes).
- `-v` affiche la sortie de DRGraph (progression, temps interne).

Une variante Python équivalente existe (`add_metagraph_layout.py`, mêmes options DRGraph) ; le binaire C est préférable sur les gros paysages.

## 4.2 Guide d'utilisation des HTML

## 4.3 Observations et limites

Dans un premier temps, il est rassurant d'observer que nos graphiques confirment les résultats de [Wagner]. Les SBN possédant la même propriété que les séquences d'ARN sur les fréquences d'apparitions de phénotypes, on retrouve bien une corrélation positive entre évolutivité et robustesse phénotypiques. [Insérer graphique scatter EP/RP en plusieurs dimensions + régression linéaire potentielle]

Malgré nos limitations à des dimensions faibles, il est possible d'observer des tendances dont l'explication pourrait éventuellement permettre de généraliser aux dimensions supérieures. [Insérer des graphiques avec des jolies flèche paint pour chaque assertion]

- La décomposition d'un SBN en sous-dynamiques distinctes est déterminante dans le calcul de ses autres propriétés. (La pluripotence n'est pas décorrélée de tout)
- Les propriétés d'un PSBN semblent être très liés à la dimension de sa plus grande hyper-face non décomposable. Notamment la variance de la robustesse génotypique pour chaque phénotype (Robustness\_std)
- [importance de SBF particulières pour former des groupes dans le graphique variance/moyenne de robustesse]

Dans le contexte de mutations sur les poids [Cf. règles de mutations] :

- Les PSBN sont plutôt proches les uns des autres, notamment au sein d'une même orientation (élément d'une orbite de permutation). Les PSBN les plus décomposés (que les permutations affectent moins[illustration]) font en quelque sorte le lien entre ces différentes orientations.

Ces conclusions résument les observations sur les visualisations générées, tentent d'en trouver des explications qui nous permettent d'oser une généralisation, mais peuvent difficilement dépasser en précision les visualisations elles-mêmes. C'est pourquoi le lecteur est fortement invité à les tester par lui-même (et qui sait, peut-être trouver de tendances que l'on aurait manquée).

## 5 Conclusion, travaux inachevés et ouverture

L'objectif principal était de montrer ce que le formalisme des SBN permettait. Et à cet égard le présent travail semble nous indiquer qu'il parvient assez bien à donner des interprétations claires de concepts généraux (comme la pluripotence), et qu'il est suffisamment pratique et riche pour que l'exploration de ses possibilités soit faisable et porteuse de sens.

Plus concrètement